🧠 Project: Arabic Word Embeddings + Text Classification

🎯 Objective

In this project, you will:

1. Train your own **Arabic Word2Vec embeddings** using `gensim`
2. Use these embeddings to build a **text classification model** in `PyTorch`

The goal is to understand how raw text becomes numerical representations and how these representations can be used in downstream NLP tasks.

🧩 Part 1: Train Arabic Word Embeddings (Word2Vec)

📌 Task

Train a Word2Vec model **from scratch** using an Arabic text corpus.

🔧 Requirements

- Use the `gensim` library
- Train either:
  - CBOW or Skip-gram
- You are free to choose:
  - Dataset (news, tweets, books, etc.)
  - Hyperparameters (vector size, window, min_count, etc.)

📍 Steps

1. **Collect Dataset**
   - Choose a sufficiently large Arabic corpus
   - Examples: news articles, Wikipedia dumps, tweets
2. **Preprocessing**
   - Normalize Arabic text (optional but recommended)
   - Tokenize sentences into words
   - Clean unwanted characters

3. **Prepare Data**

   - Format: list of tokenized sentences
     Example:

     ```
     [["أحب", "الملفات", "الجميلة"]]
     ```

4. **Train Word2Vec Model**

   - Use `gensim.models.Word2Vec`
   - Experiment with parameters:
     - `vector_size`
     - `window`
     - `min_count`
     - `sg` (0 = CBOW, 1 = Skip-gram)

5. **Evaluate Embeddings (Qualitative)**

   - Check most similar words
   - Explore semantic relationships

6. **Save Model**

   - Save embeddings for later use

🧠 Part 2: Sentence Representation

📌 Task

Convert sentences into fixed-size vectors using your trained embeddings.

💡 Idea

A simple approach:

- Represent a sentence by **averaging its word vectors**

📍 Steps

1. For each sentence:

   - Retrieve embeddings of all words
   - Ignore unknown words (or handle them)
   - Compute the average vector

Compute the average vector.

2. Build a dataset:
   - Input: sentence vectors
   - Output: class labels

🤖 Part 3: Text Classification with PyTorch

📌 Task

Train a classifier using sentence embeddings.

🔧 Requirements

- Use `PyTorch`
- Input: averaged sentence embeddings
- Output: predicted class

📍 Steps

1. **Choose Dataset**
   - Must have labeled Arabic text
   - Examples: sentiment analysis, topic classification
2. **Prepare Data**
   - Convert all sentences into vectors (from Part 2)
   - Split into:
     - Training set
     - Validation/Test set
3. **Build Model**
   - Simple architecture:
     - Input layer (embedding size)
     - One or more hidden layers
     - Output layer (number of classes)
4. **Training**
   - Define:
     - Loss function
     - Optimizer

    - Train over multiple epochs

5. **Evaluation**
    - Measure performance:
        - Accuracy (minimum)
        - (Optional) Precision, Recall, F1

📊 Deliverables

You must submit:

1. Code

- Clean and organized notebook or scripts
- Clear separation between:
    - Preprocessing
    - Embedding training
    - Classification

2. Report (Short)

Include:

- Dataset description
- Preprocessing choices
- Word2Vec parameters and reasoning
- Model architecture
- Results and observations

3. Experiments

- Try at least **2 different configurations**
    - Example:
        - CBOW vs Skip-gram
        - Different vector sizes
- Briefly compare results

💡 Bonus Ideas (Optional)

- Try weighted averaging instead of simple average

- Use TF-IDF weights
- Compare with random embeddings
- Visualize embeddings (e.g., PCA or t-SNE)

⚠️ Notes

- Do NOT use pre-trained embeddings
- Focus on understanding the pipeline
- Keep it simple but correct

🚀 Outcome

By the end of this project, you should understand:

- How word embeddings are trained
- How text is converted into vectors
- How embeddings are used in downstream tasks
- The limitations of simple sentence representation# 🧠 Project: Arabic Word Embeddings + Text Classification

🎯 Objective

In this project, you will:

1. Train your own **Arabic Word2Vec embeddings** using `gensim`
2. Use these embeddings to build a **text classification model** in `PyTorch`

The goal is to understand how raw text becomes numerical representations and how these representations can be used in downstream NLP tasks.

🧩 Part 1: Train Arabic Word Embeddings (Word2Vec)

📌 Task

Train a Word2Vec model **from scratch** using an Arabic text corpus.

🔧 Requirements

- Use the `gensim` library

- Use the `gensim` library
- Train either:
  - CBOW or Skip-gram
- You are free to choose:
  - Dataset (news, tweets, books, etc.)
  - Hyperparameters (vector size, window, min_count, etc.)

📍 Steps

1. **Collect Dataset**
   - Choose a sufficiently large Arabic corpus
   - Examples: news articles, Wikipedia dumps, tweets
2. **Preprocessing**
   - Normalize Arabic text (optional but recommended)
   - Tokenize sentences into words
   - Clean unwanted characters
3. **Prepare Data**
   - Format: list of tokenized sentences
     Example:

```
[["أحب", "الملفات"],          ["الجعوبلية"]]
```

4. **Train Word2Vec Model**
   - Use `gensim.models.Word2Vec`
   - Experiment with parameters:
     - `vector_size`
     - `window`
     - `min_count`
     - `sg` (0 = CBOW, 1 = Skip-gram)
5. **Evaluate Embeddings (Qualitative)**
   - Check most similar words
   - Explore semantic relationships
6. **Save Model**
   - Save embeddings for later use

🧠 Part 2: Sentence Representation

📌 Task

Convert sentences into fixed-size vectors using your trained embeddings.

💡 Idea

A simple approach:

- Represent a sentence by **averaging its word vectors**

📍 Steps

1. For each sentence:
   - Retrieve embeddings of all words
   - Ignore unknown words (or handle them)
   - Compute the average vector
2. Build a dataset:
   - Input: sentence vectors
   - Output: class labels

🤖 Part 3: Text Classification with PyTorch

📌 Task

Train a classifier using sentence embeddings.

🔧 Requirements

- Use `PyTorch`
- Input: averaged sentence embeddings
- Output: predicted class

📍 Steps

1. **Choose Dataset**
   - Must have labeled Arabic text
   - Examples: sentiment analysis, topic classification

2. **Prepare Data**
    - Convert all sentences into vectors (from Part 2)
    - Split into:
        - Training set
        - Validation/Test set
3. **Build Model**
    - Simple architecture:
        - Input layer (embedding size)
        - One or more hidden layers
        - Output layer (number of classes)
4. **Training**
    - Define:
        - Loss function
        - Optimizer
    - Train over multiple epochs
5. **Evaluation**
    - Measure performance:
        - Accuracy (minimum)
        - (Optional) Precision, Recall, F1

📊 Deliverables

You must submit:

1. Code

- Clean and organized notebook or scripts
- Clear separation between:
    - Preprocessing
    - Embedding training
    - Classification

2. Report (Short)

Include:

- Dataset description

- Preprocessing choices
- Word2Vec parameters and reasoning
- Model architecture
- Results and observations

3. Experiments

- Try at least **2 different configurations**
  - Example:
    - CBOW vs Skip-gram
    - Different vector sizes
- Briefly compare results

💡 Bonus Ideas (Optional)

- Try weighted averaging instead of simple average
- Use TF-IDF weights
- Compare with random embeddings
- Visualize embeddings (e.g., PCA or t-SNE)

⚠️ Notes

- Do NOT use pre-trained embeddings
- Focus on understanding the pipeline
- Keep it simple but correct

🚀 Outcome

By the end of this project, you should understand:

- How word embeddings are trained
- How text is converted into vectors
- How embeddings are used in downstream tasks
- The limitations of simple sentence representations